Programación Procedural en MySQL

Optimización y Automatización de Operaciones en Bases de Datos

Introducción a la Programación Procedural en SQL

- **SQL Procedural**
  - **Extensión del SQL estándar (declarativo) que permite lógica de programación (estructuras de control, variables, manejo de errores).**
  - **Permite crear bloques de código reutilizables y complejos que se ejecutan directamente en el servidor de la base de datos.**



Introducción a la Programación Procedural en SQL

- **Características**
  - **Reusabilidad: Define lógica una vez.**
  - **Rendimiento: Reduce el tráfico de red entre la aplicación y el servidor.**
  - **Seguridad: Control de acceso granular a la lógica de negocio.**
  - **Automatización: Ejecución automática en respuesta a eventos (Triggers).**
  - **Centralización de la lógica de negocio: Mantiene la coherencia de los datos.**

Procedimientos Almacenados (Stored Procedures)

- **Definición: Bloques de código SQL precompilados y almacenados en la base de datos. Se ejecutan bajo demanda.**
- **Características:**
  - **Pueden aceptar parámetros de entrada (IN), salida (OUT) o ambos (INOUT).**
  - **No devuelven un valor directamente, pero pueden devolver conjuntos de resultados (SELECTs) o modificar parámetros (OUT).**
  - **Optimizan el rendimiento al reducir la compilación repetida y el tráfico de red.**

Procedimientos Almacenados (Stored Procedures)

Un procedimiento almacenado es un conjunto de instrucciones, que se almacenan bajo un **nombre**

Ventajas

- Concentran lógica de negocio.
- Evitan acceso directo a las tablas.
- Reducen el tráfico de red.
- Ejecución de un procedimiento almacenado: Se invoca por su nombre
- Se pueden referenciar tablas, vistas, funciones y
- procedimientos almacenados.

Desventajas:

- Se aplican lógicas de negocios, pero no muy complejas.
- Difíciles de depurar.

Procedimientos Almacenados (Stored Procedures)

```
-- Ejemplo Procedimiento Almacenado
    DELIMITER //

    CREATE PROCEDURE ObtenerProductosPorCategoria (IN categoria_id INT)
    BEGIN
        SELECT id_producto, nombre_producto, precio
        FROM Productos
        WHERE id_categoria = categoria_id;
    END //

    DELIMITER ;

Ejecución Procedimiento
    CALL ObtenerProductosPorCategoria(1);
```

Procedimientos Almacenados (Stored Procedures)

Actividad: Crear Base de datos de prueba denominada **peliculas**

```
CREATE DATABASE peliculas;

-- Crea tabla
USE peliculas;

DROP TABLE IF EXISTS programas;

CREATE TABLE programas (
        id INTEGER NOT NULL AUTO_INCREMENT,
        titulo VARCHAR(100) NOT NULL,
        estudio VARCHAR(64) NOT NULL,
        anio integer NOT NULL DEFAULT 0,
        PRIMARY KEY(id)
) ENGINE = InnoDB;
```

Procedimientos Almacenados

```
-- Insertamos datos de prueba

insert into programas(titulo, estudio, anio) values('ET','Warner',1983);
insert into programas(titulo, estudio, anio) values('Indiana Jones','Hollywood',1987);
insert into programas(titulo, estudio) values('El Barco','Amblin');
```

Procedimientos Almacenados (Stored Procedures)

```
-- Primer procedimiento Almacenado

        delimiter //

        CREATE PROCEDURE pa_programas_lista()
        BEGIN
                SELECT *
                FROM programas;
        END
        //

        delimiter ;

-- Ejecución
        call pa_programas_lista();
```

Procedimientos Almacenados (Stored Procedures)

Ejercicios
=========
Crear un procedimiento almacenado que cuente la cantidad de programas.

Crear un procedimiento almacenado que muestre el título más antiguo

Procedimientos Almacenados (Stored Procedures)

Consideraciones

-- Delimitadores

-- Variables

-- Begin....end

```
DELIMITER // -- Cambia Delimitadores

CREATE PROCEDURE pa_programas_cantidad2()
BEGIN

        -- Declara variable local al procedimiento
        DECLARE vprogramas INT;

        SELECT COUNT(*)
        FROM programas
        INTO vprogramas;   -- Almacena resultado consulta en variable

        SELECT vprogramas; -- Muestra variable

END//

DELIMITER ; -- Recompone Delimitadores
```

Procedimientos Almacenados (Stored Procedures)

Ejercicios
==========
Agregar una columna llamada precio a la tabla programas

Crear un procedimiento almacenado que complete los valores de precio con 100

Procedimientos Almacenados (Stored Procedures)

```
-- Uso de Parámetros

DELIMITER //

CREATE PROCEDURE pa_programas_buscar(_id integer)
BEGIN
        SELECT *
        FROM programas
        WHERE id = _id;
END
//
DELIMITER ;
```

Procedimientos Almacenados (Stored Procedures)

Ejercicios
========

Crear un procedimiento almacenado que permita ingresar como parámetro una o más letras y busque todos los programas que empiecen con esas letras ingresadas.

Crear un procedimiento almacenado que actualice los precios en un porcentaje pasado como parámetro.

Procedimientos Almacenados (Stored Procedures)

-- Declaración de Variables

-- Mysql permite la declaración de variables para registrar los resultados de operaciones realizadas en la Base de Datos.

```
set @nombreVariable = valorInicial;
```

Ej:

```
set @contador = 0;
```

-- Declaramos y definimos valor para la variable contador

```
set @contador = 0;

select @contador;

select count(*)
into @contador
from programas;

select @contador;
```

Procedimientos Almacenados (Stored Procedures)

```
-- Parámetros de E/S

-- Declaramos variable de sesión e inicializamos a 0
set @contador = 0;

— Verificamos valor de la variable
select @contador;

– Ejecutamos procedimiento almacenado con parámetros de E y E/S
call pa_programas_cantidad_tipo ('serie',@contador);

– Verificamos valor de la variable
select @contador;
```

```
-- Calcular cantidad de programas.
DELIMITER //

CREATE PROCEDURE pa_programas_cantidad_tipo
(IN _tipo VARCHAR(20),  OUT _programas INT)

BEGIN
        - - Cuerpo Consulta
        SELECT COUNT(*)
        INTO _programas
        FROM programas
        WHERE tipo =_tipo;

END//

DELIMITER ;
```

Procedimientos Almacenados (Stored Procedures)

Ejercicios
==========

Crear un procedimiento almacenado que cuente la cantidad de programas por tipo y con un precio mayor que un valor dado.

Funciones Almacenadas (Stored Functions)

- **Definición: Similares a los procedimientos, pero siempre devuelven un único valor escalar.**
- **Características:**
  - **Pueden ser usadas dentro de sentencias SQL (SELECT, WHERE, etc.).**
  - **Solo aceptan parámetros de entrada (IN).**
  - **Deben ser determinísticas (devuelven el mismo resultado para los mismos inputs).**

Funciones Almacenadas (Stored Functions)

```
-- Definición función:
DELIMITER //

CREATE FUNCTION CalcularIVA (precio DECIMAL(10, 2), tasa_iva DECIMAL(5, 2))
RETURNS DECIMAL(10, 2)
DETERMINISTIC
BEGIN
    RETURN precio * tasa_iva;
END //

DELIMITER ;

-- Para usar la función:
SELECT nombre_producto, precio, CalcularIVA(precio, 0.21) AS iva_calculado
FROM Productos
WHERE id_producto = 101;
```

Funciones Almacenadas

- Las funciones son bloques de código que permiten agrupar y organizar sentencias SQL que se ejecutan al invocar la función.
- Tienen una cabecera, una sección de declaración de variables y el bloque "begin...end" que encierra las acciones.
- Una función, además contiene la cláusula "return".
- Una función acepta parámetros, se invoca con su nombre y retorna un valor.
- Puede ser invocada en una consulta.

```
DELIMITER //

CREATE FUNCTION fa_programas_cantidad()
RETURNS INT -- Especificamos parámetro de salida
BEGIN

        DECLARE vcantidad INT;

        SELECT COUNT(*)
        INTO vcantidad
        FROM programas;

        RETURN vcantidad;  -- Retornar el valor obtenido

END//

DELIMITER ;
```

Funciones Almacenadas

Ejemplo 1: Aplicación de funciones almacenadas con variables

```
set @contador = 0;

select @contador;

set @contador = fa_programas_cantidad();

select @contador;
```

Ejemplo 2: Aplicación de funciones almacenadas en sentencia SELECT

a)
```
select fa_programas_cantidad();
```

b)
```
select fa_programas_cantidad() into @contador;
```

c)
```
select tipo, fa_programas_cantidad_2(tipo)
from programas;
```

Funciones Almacenadas

Ejercicios
==========

Crear una función almacenada que cuente la cantidad de programas por tipo.

Crear una función almacenada que devuelva el precio de un programa determinado

Funciones Almacenadas

Ejercicios
==========

Crear una función almacenada que cuente la cantidad de programas por tipo.

Crear una función almacenada que devuelva el precio de un programa determinado

Triggers (Disparadores)

- **Definición: Bloques de código SQL que se ejecutan automáticamente en respuesta a un evento específico (INSERT, UPDATE, DELETE) en una tabla.**
- **Características:**
  - **Se asocian a una tabla y a un tipo de evento (BEFORE o AFTER).**
  - **Permiten implementar reglas de negocio, auditoría, replicación de datos.**
  - **Pueden acceder a los valores NEW (después del cambio) y OLD (antes del cambio) en la fila afectada.**

Triggers (Disparadores)

Ejemplo 1: Insertar en tabla logAuditoria, el detalle de operación en la tabla Productos, registrando la fecha de actualización.

```
DELIMITER //

CREATE TRIGGER after_producto_insert
AFTER INSERT ON Productos
FOR EACH ROW
BEGIN
     INSERT   INTO   LogAuditoria   (accion,
tabla, fecha)
    VALUES ('INSERT', 'Productos', NOW());
END //

DELIMITER ;
```

-- Ejemplo 2: Actualizar stock después de una venta

```
DELIMITER //

CREATE TRIGGER after_venta_insert
AFTER INSERT ON Ventas
FOR EACH ROW
BEGIN
    UPDATE Productos
    SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;
END //

DELIMITER ;
```

Triggers (Disparadores)

- Los disparadores o triggers son bloques de código que desencadenan una actividad.
- Se disparan ante las operaciones Insert, Update y Delete sobre una tabla.
- Los triggers se asocian a una tabla.
- Los triggers se utilizan para establecer reglas de gestión en la base.
- Complementan otras herramientas.
- Su funcionamiento se apoya en dos tablas auxiliares, NEW y OLD.

Ejemplo: Creamos tabla de detalle de programas

```
CREATE TABLE prog_detalle (
        id_det INTEGER NOT NULL AUTO_INCREMENT,
        id VARCHAR(100) NOT NULL,
        titulo varchar(255),
        entradas  integer,
        recaudacion double(5,2),
        PRIMARY KEY(id_det)
) ENGINE = InnoDB;
```

Triggers (Disparadores)

```
DELIMITER //

CREATE TRIGGER tr_programa_insertar -- Nombre trigger
AFTER INSERT ON programas           -- Nombre Tabla a la que se asocia el trigger
FOR EACH ROW
BEGIN
        insert into prog_detalle(id, titulo,entradas, recaudacion) values (NEW.id, NEW.titulo,'Sin datos', 0);
END
//

DELIMITER ;

– Al ejecutar este INSERT, se dispara el trigger
insert into programas(titulo, estudio, anio) values('Ghost','Amblin',1985);
```

Triggers (Disparadores)

```
DELIMITER //

CREATE TRIGGER tr_programa_actualizar       -- Nombre
AFTER UPDATE ON programas                  -- Tabla
FOR EACH ROW
BEGIN
        update prog_detalle set titulo=NEW.titulo where id = NEW.id;
END

//
DELIMITER ;

– Al ejecutar este UPDATE, se dispara el trigger
update programas set titulo = 'Titanic' where id = 7;
```

Triggers (Disparadores)

```
DELIMITER //

CREATE TRIGGER tr_programa_borrar    -- Nombre
AFTER DELETE ON programas            -- Tabla
FOR EACH ROW
BEGIN
        delete from prog_detalle where id = OLD.id;
END

//
DELIMITER ;

– Al ejecutar este DELETE, se dispara el trigger
delete from programas set titulo = 'Titanic' where id = 7;
```

Control de Flujo - IF .. THEN .. ELSE

```
delimiter //
create procedure p1(in p1 int)        /* Parámetro de entrada */
        begin
                declare miVar int;        /* se declara variable local */
                set miVar = p1 +1 ;       /* se establece la variable */
                if miVar = 12 then
                        select 'verdadero' ;
                else
                        select 'falso' ;
                end if;
        end;
//
delimiter ;
```

Control de Flujo - CASE

```
delimiter //
create procedure p2 (in p1 int)
        begin
                declare var int ;
                set var = p1;
                case var
                        when 2 then select 'opción 2';
                        when 3 then  select 'opción 3';
                        else  select 'opción N';
                end case;
        end;
//

delimiter ;
```

Control de Flujo

Ejercicio

Crear un procedimiento almacenado que permita determinar si dos cadenas de caracteres son iguales.

Control de Flujo - WHILE

```
delimiter //
create procedure p3()
        begin
                declare v int;
                set v = 0;
                while v < 5 do
                        select v;
                        set v = v +1 ;
                end while;
        end;
//
delimiter ;
```

Control de Flujo - REPEAT

```
delimiter //
create procedure p4()
        begin
                declare v int;
                set v = 20;
                repeat
                        select v; --insert into lista values(v);
                        set v = v + 1;
                        until v >= 1
                end repeat;
        end;
//
delimiter ;
```

Control de Flujo - LOOP

```
delimiter //
create procedure p5()
        begin
                declare v int;
                set v = 0;
                loop_label : loop
                        insert into lista values (v);
                        set v = v + 1;
                        if v >= 5 then
                                leave loop_label;
                        end if;
                end loop;
        end;
//
delimiter ;
```